

Advanced Operating Systems: SC

Exam correction

Exercise 1: Course Questions (6 points)

Question 1) Answer TRUE or FALSE

- a) As long as a process is blocked in a semaphore's waiting queue, it is engaged in busy waiting. **False (1pt)**
- b) A binary semaphore (or mutex) is used to manage a variable number of shared resources. **False (1pt)**

Question 2) When can Fork/Join be deterministic? (2pt)

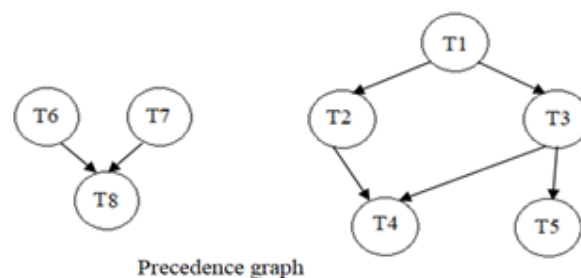
- **No shared variables (or read-only shared variables)**
- **No data dependencies between parallel tasks**
- **Commutative and associative operations on combined results**

Question 3) What are Coffman's four necessary conditions for a deadlock to occur?. **See course (2pts)**

Exercise 2: (8 points)

Question 1) Are the following words part of the language associated with the system defined by the precedence graph below? Justify your answer if the response is no.

- a) b1 b6 e1 b2 e6 b3 b7 e3 e2 b4 e7 b8 e4 b5 e5 e8
- b) b6 b7 e6 b1 e1 e7 b2 e2 b3 e3 b4 b8 e4 b5 e5 e8
- c) b1 e1 b2 b3 b7 e2 b6 b4 e3 b5 e7 e6 b8 e8 e5 e4



- a) **b1 b6 e1 b2 e6 b3 b7 e3 e2 b4 e7 b8 e4 b5 e5 e8**

Vérification des contraintes :

- **e1<b2 ✓**
- **e1<b3 ✓**
- **e2<b4 ✓**

- $e_3 < b_4$ ✓
- $e_3 < b_5$ ✓
- $e_6 < b_8$ ✓
- $e_7 < b_8$ ✓

✓ All constraints are satisfied $\Rightarrow$ This word is part of the language. (1pt)

b) $b_6 b_7 e_6 b_1 e_1 e_7 b_2 e_2 b_3 e_3 b_4 b_8 e_4 b_5 e_5 e_8$

Verification :

- $e_1 < b_2, e_1 < b_3$ ✓
- $e_2 < b_4$ ✓
- $e_3 < b_4, e_3 < b_5$ ✓
- $e_6 < b_8$ ✓
- $e_7 < b_8$ ✓

✓ All constraints are satisfied $\Rightarrow$ This word is part of the language. (1pt)

c) $b_1 e_1 b_2 b_3 b_7 e_2 b_6 b_4 e_3 b_5 e_7 e_6 b_8 e_8 e_5 e_4$

Let's analyze:

- e_1 before b_2, b_3 ?
 - Yes: ✓
- b_4 before e_3 ?
 - No: ✓

✗ No! $b_4 < e_3$ then according to the precedence graph $T_3 < T_4$ (1pt)

Question 2) The following expressions describe the precedence relations — both parallel and sequential — among six processes P1 to P6:

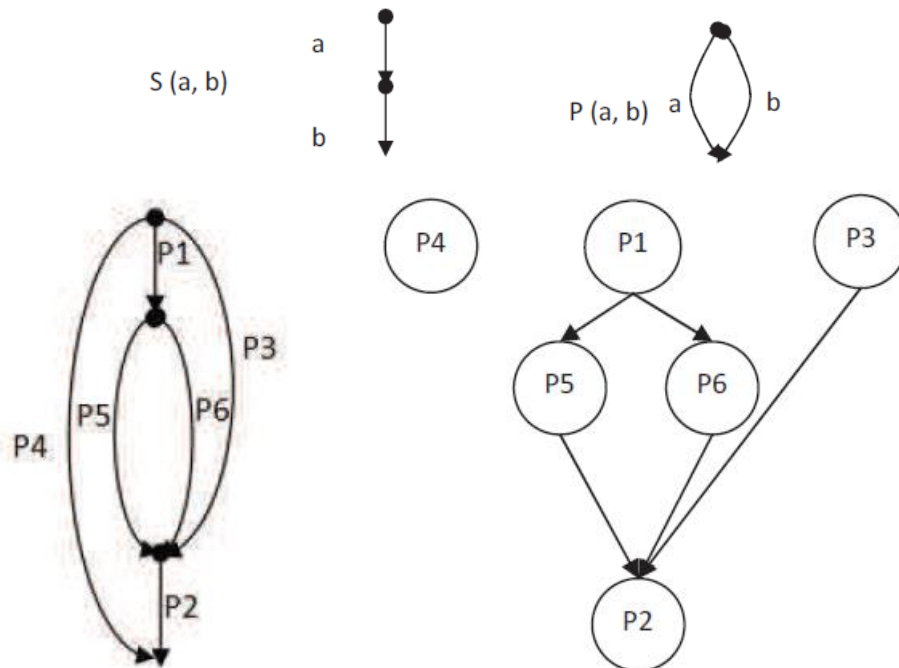
$$P(S(P(P_3, S(P_1, P(P_6, P_5))), P_2), P_4)$$

Transform this expression into a program using:

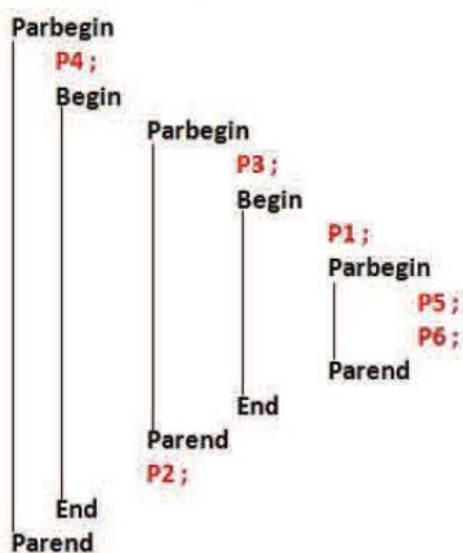
- parbegin/parend** (maximizing parallelism)
- The primitives: **fork** and **join**.

Note:

- $S(a, b)$ represents the **sequential execution** of processes a and then b
- $P(a, b)$ represents the **parallel execution** of processes a and b



a. Parbegin/Parend (2.5pt)



b. Fork, Join et Quit (2.5pt)

```
Count2 = 3 ;
Fork L1 ;
P4 ;
Quit ;
L1 : Fork L2 ;
P1 ;
Fork L3 ;
P5 ;
Goto L4 ;
L3 : P6 ;
Goto L4 ;
L2 : P3 ;
L4 : Join Count2 ;
P2 ;
```

Exercise 3: (6 points)

Consider the following resource allocation system :

Process	Allocation R1 R2 R3 R4	Max R1 R2 R3 R4	Available R1 R2 R3 R4
P1	0 0 0 0	4 2 3 1	5 2 1 1
P2	0 1 2 2	2 1 4 4	
P3	5 1 1 1	5 1 2 2	

a) What is the total number of instances of each resource?

Answer (a): (2pts)

- R1 total = 10
- R2 total = 4
- R3 total = 4
- R4 total = 4

b) Is the system in a safe state?

Yes, the system is in a safe state. Safe sequence: P3 → P2 → P1. (1pt)

c) Apply the Banker's Algorithm to the following requests:

- 1) P1 requests 2 instances of R4.
- 2) P3 requests 1 instance of R3.
- 3) P2 requests 1 instance of R4.

1. P1 requests 2 instances of R4: Denied (exceeds Need). (1pt)
2. P3 requests 1 instance of R3: Granted (safe sequence: P3 → P2 → P1). (1pt)
3. P2 requests 1 instance of R4: Denied (unsafe state). (1pt)